

Component S: Teaching a Computer to Guess How a Forest Grows, Without Rerunning the Forest

A plain-language guide to a statistical emulator of forest demography
and tree-trait diversity in a hybrid Earth System Model

Abstract

This report explains, without assuming any background in statistics, computer programming, or machine learning, how a piece of climate-modeling software called “Component S” works and how well it works. Component S is a small part of a much larger effort to build faster computer models of how the land surface of our planet – especially its forests – behaves inside global climate simulations. Rather than simulating every tree in a forest, tree by tree, day by day, which is accurate but extremely slow, Component S is a trained statistical model that has learned to guess, once per year, how many trees of each kind survive or newly sprout in a patch of forest, and what physical traits those trees have – things like leaf thickness, wood density, and rooting depth. It was trained on millions of simulated trees produced by a much slower, highly detailed forest simulator, and it is tested by hiding some locations from it during training and then checking how well it predicts those unseen locations. This report walks through where the training data comes from, how the two-part prediction method works, how the testing is done, what each accuracy number actually measures in plain terms, what the current results show, and where the model still needs improvement. It also describes several mistakes the project caught in itself along the way, as evidence of a careful, skeptical checking process rather than something to be hidden. Two things are worth flagging up front, and are repeated where relevant: Component S currently only handles trees (grasses are handled by a different part of the wider model and are not discussed further here), and the specific accuracy numbers in this report describe how well Component S reproduces one year’s worth of change at a time, offline, against recorded outcomes – not how it behaves when it is run repeatedly, year after year, inside a live coupled simulation, which is a related but separate question still being worked on.

Contents

1	Introduction and Motivation	3
1.1	What is an Earth System Model, and why does the land need its own model inside it?	3
1.2	Why not just run the full, detailed forest simulation every time?	3
1.3	Where Component S fits, one more time, concretely	4
2	The Training Data	4
2.1	The “ground truth” is a simulation, not a field survey	4
2.2	Spatial and temporal scale	5
2.3	Why the same history was simulated twice, with two different “seeds”	5
2.4	What gets recorded, per tree, per year	6
2.5	The environmental “clues” Component S is given	6
2.6	The overall scale of the training data	7
3	How Component S Makes Predictions	7

3.1	Predicting counts: a forest of decision trees	8
3.1.1	What a single decision tree is	8
3.1.2	Why many trees, not just one, and what “random” does for us here	8
3.1.3	Getting a whole <i>range</i> of possible answers, not just one number	9
3.2	Assigning traits to new trees: the copula	9
3.2.1	The problem the copula solves	9
3.2.2	How a copula works, step by step, without symbols	9
3.2.3	Which traits the copula actually assigns	10
4	How We Test Whether It Works	11
4.1	Testing on locations the model has never seen: held-out-by-location testing . . .	11
4.2	The natural noise floor: what seed1 versus seed2 tells us	12
4.3	Testing on a climate the model has never seen: the future-climate hold-out	13
5	Understanding the Numbers: What Each Measure Means	13
5.1	R-squared (written R^2)	13
5.2	Root-mean-square error (RMSE)	13
5.3	The Kolmogorov-Smirnov comparison (a whole-distribution comparison)	14
5.4	Wasserstein (“earth-mover”) distance – a concept worth knowing, though not currently computed in this project	14
5.5	Interquartile-range-normalized root-mean-square error (nqrmse)	15
5.6	The prediction-spread ratio	15
6	Results: What the Validation Actually Shows	16
6.1	Tree-count prediction accuracy	16
6.2	Trait prediction accuracy – the four live axes	17
6.3	Biomass and tree size – diagnostic-only, and close to the achievable ceiling in this offline check	19
6.4	Overall gate result	21
7	Why We Should Trust These Results	21
7.1	The “which trees count” bug	22
7.2	The “basis_ratio” explanation that sounded right but wasn’t	23
7.3	Rows that silently changed count on their own	24
8	Current Limitations and Ongoing Work	25
9	Conclusion	27
	Glossary	29

1 Introduction and Motivation

1.1 What is an Earth System Model, and why does the land need its own model inside it?

An Earth System Model is a large computer program that simulates the entire climate system of our planet at once: the atmosphere (winds, clouds, temperature), the oceans, the ice sheets, and the land surface, all talking to each other continuously as the simulation runs forward in time, sometimes for centuries. The land surface matters enormously to this system because forests and other vegetation absorb carbon dioxide from the air, release water vapor back into the atmosphere through their leaves, and change how much sunlight the ground reflects versus absorbs. If you want to predict how the climate will change over the next hundred years, you cannot ignore what the forests are doing – and forests are not static backdrops; they grow, they die, they shift which species dominate a location as the climate around them changes.

Because of this, an Earth System Model needs a “land component”: a sub-model, running inside the bigger simulation, whose job is specifically to track vegetation and soil. This report is about one such land component, currently under development, which is built from three cooperating pieces, each with a short, memorable code name:

- **F**, the *fast* component: a daily physical simulation core that tracks photosynthesis (the chemical process by which plants use sunlight to build sugars, absorbing carbon dioxide from the air and releasing oxygen), water use, and carbon movement through the plant and soil, one day at a time. It is called “fast” because it uses efficient, direct physical calculations, and it is built so that a mathematical technique called differentiation can be applied to it (this lets other software components learn how to adjust it automatically – a detail not needed for understanding this report, but part of why F is engineered the way it is).
- **E**, the *energy* component: a model of how energy balances at the land surface – how the ground and canopy heat up in the sun, cool down at night, and exchange heat with the air above – producing a realistic skin temperature (the temperature right at the surface of the leaves or soil, as opposed to the air a meter or two above it).
- **S**, the *slow* component – the subject of this report. “Slow” refers to the fact that forest demographic change (which trees are born, which die, how the mix of species and sizes shifts) happens gradually, over full years, in contrast to F’s daily physical calculations. Component S’s job is to predict, once per simulated year, how many trees of each type survive or newly establish in a patch of land, and what physical traits (**leaf economy** – a general term biologists use for the trade-off between building cheap, fast, short-lived leaves versus expensive, tough, long-lived ones – wood density, rooting depth, drought tolerance) those trees carry. As currently built, Component S predicts trees only; grasses in the same patch are handled by a different part of the wider model (F), and extending Component S to also cover grass demography is a separate, not-yet-started piece of future work, mentioned again in Section 7.

Together, F handles the day-to-day physics, E handles the surface energy and temperature, and S handles the yearly bookkeeping of which trees exist and what they are like. This report focuses entirely on S.

1.2 Why not just run the full, detailed forest simulation every time?

There already exists a highly detailed, physically faithful computer simulation of vegetation called LPJmL-FIT. It is what computational scientists call a “process-based” model: rather than being trained on data, it directly encodes the biological and physical processes governing plant growth

– photosynthesis, respiration (the process by which living tissue burns stored sugars to release energy, consuming oxygen and producing carbon dioxide, the reverse of photosynthesis), water uptake, competition for light and water between neighboring trees, and death from starvation, old age, drought, or cold stress. Crucially for this project, LPJmL-FIT can be run in a mode where it tracks every individual simulated tree separately, rather than averaging a forest stand into one blended value. This makes it extremely realistic, but also extremely slow: simulating every tree, every day, for every one of tens of thousands of locations on the globe, for decades or centuries, is far too computationally expensive to run inside an Earth System Model that itself needs to simulate the atmosphere and ocean simultaneously, often many times over for different scenarios.

This is the core motivation for building Component S. The idea is to run the full, slow, detailed simulation once, offline, over a large number of representative locations and years, and use its output as a kind of textbook of examples: “given this climate and this starting forest, here is what happened one year later.” A separate, much cheaper statistical model is then trained on that textbook to reproduce the same year-to-year behavior, without re-deriving it from physical first principles every time. Once trained, this cheaper model can be plugged into the full Earth System Model and consulted every simulated year in a small fraction of the time the original detailed simulation would have taken – while still being continuously checked against the detailed simulation’s own output to make sure it has not drifted away from something realistic.

This general strategy – training a fast statistical stand-in for an expensive physical simulation – is sometimes called “emulation,” and the resulting stand-in model is called an “emulator.” Component S is, specifically, a *statistical emulator of forest demography and tree-trait diversity*: demography meaning the study of birth, growth, and death in a population (here, a population of trees), and tree-trait diversity meaning the natural variety of physical characteristics – leaf thickness, wood density, rooting depth, drought tolerance – found among the individual trees that make up a real forest.

1.3 Where Component S fits, one more time, concretely

Every simulated year, in the coupled system, the sequence looks roughly like this: F runs day by day through the year, producing a running tally of things like total carbon gained through photosynthesis and how much water stress the vegetation experienced; E keeps the surface energy balance consistent throughout; and once the year is complete, S is consulted exactly once, taking that year’s summary numbers as input and producing two outputs – how many trees of each type now exist in the patch (having gained new individuals through establishment and lost some to mortality), and what physical traits the newly established trees have. Those updated numbers then feed back into F for the following year, and the cycle repeats. This report is about how S makes that once-a-year prediction, and how confident we can be that its predictions resemble what the detailed physical simulation would actually have produced. Everything reported in Sections 6 and beyond describes how well S reproduces one such yearly step when checked directly against recorded outcomes; how S behaves when chained together, year after year, inside a live running coupled simulation is a related but distinct question that this report does not attempt to answer (Section 7 says more about this).

2 The Training Data

2.1 The “ground truth” is a simulation, not a field survey

It is important to be precise about what Component S is trained on. The “ground truth” used to train and test it is **not** data collected from real forests with tape measures and increment borers. It is the output of the detailed physical simulator, LPJmL-FIT, described above, run in

its most detailed “individual” mode, where every simulated tree is tracked separately rather than averaged into a stand-level summary. So Component S is learning to imitate another computer model, not learning directly from nature – but that other computer model was itself built and calibrated to reproduce real forest physiology and dynamics, so imitating it well is a meaningful and useful goal, and it is the only source of data detailed and complete enough (every tree, every year, everywhere) to train a model like this.

2.2 Spatial and temporal scale

The detailed simulation was run across a grid covering the entire land surface of the globe, divided into **67,420 grid cells**, each representing a real-world location (identified by its latitude and longitude). Not every one of these 67,420 locations ends up contributing to Component S’s training data – some are ice, desert, or otherwise have no trees of the tracked types in a given period – so the tables used to train and test the model, described later in this section, cover a large majority of the 67,420 cells rather than every single one; the exact count differs slightly from table to table depending on exactly which years and which trees are involved, and this report will flag the relevant count each time a specific table is discussed. One example location used throughout the project’s own internal testing is a mixed beech forest in Hainich, Germany, referred to by its grid position as cell number 42,490.

Two overlapping time periods were simulated:

- A **historical** period, the years 2000 through 2019, representing the climate as it actually occurred.
- A **future-climate** period beginning in 2020, following a standard climate-science warming trajectory used across the field, often referred to by the shorthand “SSP370” (this is simply the name of one specific, widely used set of assumptions about how much greenhouse gas humanity emits going forward – the exact number and letters are a label, not something the reader needs to decode further).

Having both a historical period and a future, warmer period matters because it lets the project check whether Component S has learned something general about how climate affects forest demography, or whether it has merely memorized quirks of one particular climate period (more on this in Section 4).

2.3 Why the same history was simulated twice, with two different “seeds”

Here is a subtlety that turns out to matter a great deal for how the results in this report should be interpreted. The detailed simulator LPJmL-FIT is not perfectly repeatable, even when given the exact same climate inputs for the exact same location. Internally, it uses a source of randomness (a “random number generator,” the same basic idea as a computer rolling dice) to decide, for example, the order in which competing trees in a crowded patch get first access to available water and light each day. Which numbers that internal dice-roller produces depends on a starting number called a “seed” – the same idea as shuffling a deck of cards: if you start from a different arrangement (seed) and follow the identical shuffling procedure, you get a different final order, even though the procedure itself never changed.

Because of this, the project ran the entire historical period **twice**, changing nothing except this internal seed, producing two data sets referred to as “seed1” and “seed2.” Both are equally valid, equally realistic simulations of the exact same location under the exact same climate – they simply represent two different possible ways the day-to-day competition could have played out by chance. This turns out to be extremely useful for testing, because comparing seed1 against seed2 tells us how much two runs of the *same* true model naturally disagree with each other, purely from that internal randomness, even with zero involvement from Component S.

That comparison sets a realistic ceiling on how well any environment-driven predictor could ever match either individual run (Section 4.2 explains exactly how this ceiling is used).

2.4 What gets recorded, per tree, per year

For every living tree, in every small simulated forest plot (called a “patch”), in every calendar year, the detailed simulation records a row of information – twenty-nine separate pieces of information in total. It is a very rich record precisely because Component S needs to learn not just how many trees survive, but the full texture of how their traits and vital signs relate to one another. It is easiest to follow if grouped into the categories a biologist would naturally reach for – think of it as what a very thorough forest-inventory technician would write on a clipboard for every single tree, every single year, if given unlimited time and equipment:

- **Identity and bookkeeping:** the calendar year; a unique identifier for that tree within its patch; which patch and which of the 67,420 global grid cells it belongs to; which of seven tree species-groups it belongs to (for example, tropical evergreen broadleaf, or the temperate beech type found at the Hainich test site).
- **Size and age:** its height; its age in years since it started growing; its above-ground biomass (the mass of the woody and leafy material above the soil, essentially how much “tree” there is up top); its total vegetation carbon (above- and below-ground combined).
- **This year’s carbon and water budget:** how much water it transpired that year (transpiration is the process by which a plant draws water up from its roots and releases it as vapor through tiny pores in its leaves, in order to keep its photosynthesis chemistry running – this is the main way vegetation moves water from soil into the atmosphere); how much carbon it captured through photosynthesis, both the gross amount before subtracting the plant’s own respiration costs and the net amount afterward; a score from zero to one describing how much water stress the tree experienced that year, where low values mean the tree was drought-stressed.
- **Inherited physical traits:** specific leaf area (a measure of how much leaf surface area the tree builds per unit of leaf mass – thin, cheap, fast-growing leaves have a high value, thick, tough, long-lived leaves have a low value); an expected organ lifespan trait (roughly, how long the tree’s leaves and other organs are built to keep functioning before being shed or replaced – part of the same “leaf economy” trade-off just introduced); wood density (how much mass is packed into a given volume of wood – denser wood generally resists drought and structural damage better but grows more slowly); leaf area index (a measure of how much leaf surface area is layered over the ground beneath this individual’s crown); the fraction of ground this tree’s crown shades; a drought-tolerance trait describing the minimum water-stress level the tree can survive; two measures of rooting depth (one realized, meaning how deep the roots actually grew given constraints, and one potential, meaning the depth the tree’s genetics would allow if unconstrained); two further parameters describing the mathematical shape of its root profile.
- **This year’s fate:** four separate probabilities of death that year broken out by cause – starvation (insufficient carbon income), old age, drought, and temperature stress – a combined overall mortality probability, and a yes/no flag for whether the tree actually died that year.

2.5 The environmental “clues” Component S is given

Component S does not have to guess blind. Each year, it is handed a summary of what happened physically in that patch that year, produced by the faster daily physical component F. These summary clues include: the total yearly carbon gain of the patch; a measure of growth efficiency

(how much growth was achieved per unit of leaf area, which indicates whether the vegetation is thriving or straining); a measure of how severe drought stress was that year; root-zone soil moisture, expressed as a fraction of how much water the soil could possibly hold; the average height of the trees in the patch, weighted by how much canopy each occupies; the height of the tallest tree; the total biomass of the whole patch; the total leaf area index of the whole standing forest (the same kind of measurement introduced in Section 2.4 for a single tree’s own crown, but now added up across every tree in the patch); the total fraction of ground shaded by canopy, capped so it cannot exceed complete coverage; the average age of the standing trees at the start of the year; and how many living trees the patch had the previous year (giving the model a form of memory of where the forest stood before this year’s changes). One of these clues – the drought-stress-severity measure – is built and maintained by a different part of the wider modeling project than the team responsible for Component S itself, and is currently known to disagree, by a substantial and still-unresolved margin, with an independent internal check of the same quantity; this is listed again as an open item in Section 7.

2.6 The overall scale of the training data

Putting the spatial coverage, the twenty years, and the two random re-runs together: the patch-year table used to train the tree-count model covers 53,699 of the 67,420 global grid cells for the historical period and 58,496 cells for the future-climate period, together totaling roughly 121 million patch-year rows (one row per patch per year – not one row per individual tree). The underlying per-tree records that feed this table, and the separate trait-prediction model, are considerably larger still: on the order of 198 million individual tree-year records for the historical period and around 829 million for the future-climate period, across all patches and years. The very first, most detailed raw version of this data occupies roughly forty-six gigabytes of storage. This scale is part of why a carefully engineered, fast statistical model – rather than the original slow simulator – is the only practical way to make this information usable inside a global climate simulation.

3 How Component S Makes Predictions

Component S’s job splits naturally into two separate questions, answered by two separate statistical techniques working in sequence.

1. **How many** new trees establish in a patch this year, and how many existing trees of each kind survive? This is answered by a technique called a **forest of decision trees** (an unfortunate but coincidental naming collision – this is a general-purpose statistical technique that happens to be called a “forest,” and has nothing to do with which numbers describe an actual forest; the coincidence is worth being aware of so the reader is not confused later).
2. Given that some number of new trees are establishing, **what combination of traits** does each new tree get – how leafy, how dense its wood, how deep its roots, how drought-tolerant? This is answered by a separate technique called a **copula** (explained fully in Section 3.2).

These two questions are genuinely separate because Component S’s built-in mortality rule (the rule governing which trees die) does not look at a tree’s individual traits at all – every tree in a cohort is thinned by the same fraction regardless of whether it happens to have thick or thin leaves, dense or light wood. This means that, unlike in the real world (where a drought might kill drought-intolerant trees preferentially, changing the community’s trait makeup through selective survival), in this emulator the entire trait makeup of the forest is decided *only* by what gets planted at establishment. So getting the establishment count right says nothing on its own about

getting the trait makeup of those new trees right – a second, dedicated mechanism is needed, and that mechanism must respect the fact that a real tree’s traits are not independent of one another (a tree with very dense wood tends to also have particular, correlated tendencies in its other traits, because these are not arbitrary, unrelated dials but jointly inherited characteristics).

3.1 Predicting counts: a forest of decision trees

3.1.1 What a single decision tree is

Imagine you are trying to guess how many new trees will sprout in a forest patch this year, and you are handed a stack of environmental clues about the patch: how much carbon it gained, how severe the drought was, how old the existing trees are, and so on. A **decision tree** is a simple, mechanical way to use those clues: it asks a sequence of yes-or-no questions, one at a time, each of the form “is this particular clue above or below some threshold?” – for instance, “was this year’s drought severity above a certain level?” Depending on the answer, you are routed down one branch or another to the next question, and so on, until you reach an end point (called a “leaf,” by analogy with a tree’s branching structure) where no more questions are asked. Every leaf has been reached by a specific combination of yes/no answers, which means every training example that ends up at that same leaf shares a similar set of circumstances – similar drought severity, similar existing forest age, and so on. The leaf’s prediction is simply the average of however many training examples (real, previously simulated patch-years) happened to land there. Building the tree means choosing, at every branching point, whichever yes/no question does the best job of separating the training examples into groups that are as internally similar (and as different from each other) as possible – imagine sorting a pile of mixed nuts by repeatedly asking the single most useful yes/no question at each step, until each final pile is fairly uniform.

3.1.2 Why many trees, not just one, and what “random” does for us here

A single decision tree, built this way, tends to overfit – meaning it can end up memorizing quirks specific to the particular training examples it happened to see, rather than learning something that generalizes to new, unseen situations. The standard fix, and the one Component S uses, is to build not one tree but many – in Component S’s actual production setup, **one hundred and fifty** separate trees – and average their predictions together. Crucially, each of the one hundred and fifty trees is not built from the identical training data in the identical way, in two separate ways.

First, each tree is grown from its own random resample of the training examples (a technique called “bootstrapping”: imagine you have a bag of one hundred marbles representing one hundred training examples, and to build one tree you reach in and draw one hundred marbles *with replacement* – meaning after you look at a marble you put it back before drawing again, so by chance some marbles get drawn several times and others not at all; you then use that particular resampled bag to grow one tree).

Second, at every single yes/no branching question, each tree is only allowed to consider a randomly chosen subset of the available clues, rather than all of them. An analogy helps here: imagine asking one hundred and fifty different doctors to diagnose the same patient, but each doctor is only allowed to look at a handful of the patient’s test results, chosen at random just for them. No single doctor sees the whole picture, so no one doctor can fixate on the one most obvious-looking symptom and build their entire diagnosis around it; pooling all one hundred and fifty partial opinions together ends up more reliable than even one doctor who saw everything, because the pooled opinion cannot get stuck on any one clue’s blind spots.

The effect of both kinds of randomness together is that the one hundred and fifty trees end up disagreeing with each other in their individual quirks and mistakes, but agreeing on the genuinely

useful patterns – so averaging their predictions cancels out much of the individual overfitting while keeping the real signal. This overall technique – many randomized decision trees, averaged together – is widely known in the statistical world as a “random forest,” and that is the “forest” in this component’s method, distinct from the biological forest it is being asked to predict.

3.1.3 Getting a whole *range* of possible answers, not just one number

An ordinary random forest, used in its simplest form, produces one averaged number per prediction. Component S’s version is built to do something a bit richer: rather than throwing away the individual training examples that land in a leaf once their average is computed, it can optionally keep every single one of them. Then, to make a prediction for a brand-new situation, instead of reporting just the average, it gathers up *all* of the individual raw training values that ended up in the relevant leaf of every one of the one hundred and fifty trees, pools them together, and treats that entire pooled collection as the model’s answer for “what range of outcomes is plausible here.” If you want a single best guess, you can still take the average or the middle value of that pool; but if you want to know how uncertain the guess is, or what an unusually high or unusually low outcome would look like, you can ask the pool directly – for instance, “what value falls right in the middle of this pool” or “what value has ninety-five percent of the pool below it.” This full pooled collection of plausible values, for a given situation, is what statisticians call a **distribution** – not a single number, but the entire shape of what is more or less likely. Think of it like asking not just “what temperature will it be tomorrow” but “show me the entire range of temperatures that have historically occurred on days with this exact weather pattern, from the coldest such day on record to the warmest.” That full spread is far more informative than a single average, especially when the underlying process (here, tree establishment, which depends partly on chance) is naturally variable rather than perfectly predictable.

3.2 Assigning traits to new trees: the copula

3.2.1 The problem the copula solves

Suppose the decision-tree forest above has just told us: “this patch, this year, gains twelve new trees.” We still need to decide what those twelve trees are like – their leaf economy, wood density, rooting depth, and drought tolerance. We could, in principle, look up each trait’s own typical range separately and pick a value for each trait independently, with no regard for the others. But this would be biologically wrong, because in reality these traits are not independent: a tree with unusually dense wood tends, for genuine biological reasons baked into how the original simulator assigns these traits, to also tend toward particular values of its other traits. If we picked each trait separately and ignored these tendencies, we would end up inventing biologically implausible trees – for example, giving a tree an extremely light, fast-growing leaf type together with an extremely dense, slow-growing wood type, a mismatch that essentially never occurs together in the training data. A **copula** is the statistical tool used specifically to draw several related quantities together, in a way that respects how they tend to move together, while still keeping each individual quantity’s own realistic range of values intact.

3.2.2 How a copula works, step by step, without symbols

The recipe has three steps.

Step one – convert each trait to a percentile rank. For each trait separately, using the training data described in Section 2, instead of asking “what is the raw value,” the recipe first asks “how unusual is this value, on a scale from zero to one hundred percent, compared to everything else observed for this trait at this kind of location?” For instance, a wood density value might sit at the seventieth percentile, meaning seventy percent of comparable trees have lower wood density and thirty percent have higher. This step matters because it puts every trait

– no matter what units or typical range it naturally has – onto the same common zero-to-one-hundred percentile scale, which is what makes it possible to describe how traits move together in the next step. At the end of Step One, we have, for each trait, the whole realistic spread of percentile values seen across the training data.

Step two – decide, together, where the new tree lands on that percentile scale for all four traits at once. Rather than choosing each new tree’s percentile rank for each trait on its own, the recipe draws all four percentile ranks *together*, using a shared pattern of relationships (called a **correlation** pattern) learned from the training data. Correlation, in plain terms, is simply the tendency of two things to rise and fall together (a positive correlation) or move in opposite directions (a negative correlation) – the same everyday idea as noticing that, across a class of students, taller students also tend to have larger shoe sizes: knowing one gives you a useful hint about the other, even though it is not a perfect rule. Concretely, the technique proceeds in two moves. First, it draws four ordinary, independent random numbers from the familiar bell-curve pattern (a “standard normal” distribution – the classic symmetric hump shape where values near the middle are common and extreme values in either direction are rare, the same shape that height or measurement error typically follows). Second, it mathematically *re-mixes* those four independent numbers into a jointly correlated batch, according to the learned correlation pattern, using a piece of matrix arithmetic called a “Cholesky factorization” (the reader does not need to understand the arithmetic itself – only that its role is to take independent bell-curve numbers and re-mix them into a jointly correlated set that matches a target correlation pattern, in the same spirit as tilting a table so that marbles that were rolling independently start rolling together in a coordinated diagonal direction). The result of this step is four correlated percentile ranks for the new tree – one per trait – that, once translated onto the zero-to-one-hundred percentile scale, tend to be jointly high or jointly low in exactly the pattern the real trees exhibit: for instance, an unusually high wood-density percentile tending to arrive packaged together with an unusually low leaf-area percentile, matching the genuine biological trade-off between these two traits.

Step three – translate each percentile rank back into a real, physical value. Finally, each trait’s correlated percentile rank is converted back into an actual physical number – an actual wood density in its normal units, an actual leaf-area value in its normal units – using that specific trait’s own observed range of values for this location and climate (this reuses the same pooled-distribution machinery from the decision-tree forest, described in Section 3.1.3: whatever value sits at, say, the seventieth percentile of the pooled training values for wood density at this kind of location becomes the answer). The overall effect of these three steps is a set of newly assigned traits that are correlated the right way, exactly as real trees are, while each individual trait still falls within its own realistic, location-appropriate spread of values.

3.2.3 Which traits the copula actually assigns

Four traits are handled this way as genuine, working parts of the running model: specific leaf area, wood density, the potential rooting-depth trait – internally labeled with the short code “D95max,” meaning the depth within which ninety-five percent of a tree’s root system sits, given unconstrained growth – and the minimum-water-stress tolerance trait – internally labeled “minwscal,” meaning how much drought stress a tree can withstand before its growth or survival is compromised. These two short codes, D95max and minwscal, are simply the internal shorthand names carried through into the results tables in Section 6; they do not need to be decoded any further than the plain-language descriptions just given.

Two further quantities – above-ground biomass and tree height – are also computed through this same copula machinery, but purely to check the model’s realism; they are not fed back into the running simulation as newly assigned traits. This distinction matters for a specific

engineering reason: biomass and height are not traits an individual is born with at the moment it establishes – they are outcomes that emerge afterward, from years of growth governed by the fast physical component F. Treating them as establishment traits would have meant double-counting something the physical simulation already computes on its own. Also, changing which traits get formally saved as part of the model’s core output would have disrupted a frozen agreement other parts of the wider modeling system depend on. The project confirmed, by direct comparison, that adding these two extra diagnostic-only measurements changes absolutely nothing about the four real, saved trait predictions – the files produced with or without the diagnostic measurements turned on come out byte-for-byte identical, and the four core trait predictions are unaffected either way. This is reassuring: the extra bookkeeping used to double-check biomass and height carries zero risk of quietly breaking anything else. It is important to note, however, that “checking the model’s realism” here means checking it offline, one year at a time, against recorded outcomes – not checking how biomass and height would evolve if Component S were run repeatedly, year after year, inside a live coupled simulation (Section 6.3 returns to this distinction).

4 How We Test Whether It Works

Building a model that fits the data it was trained on is the easy part – almost any sufficiently flexible model can be tuned to reproduce examples it has already seen. The genuinely hard and meaningful question is whether the model can make good guesses about situations it has *never* seen. Component S is tested against exactly this harder standard, using two complementary strategies.

4.1 Testing on locations the model has never seen: held-out-by-location testing

The core testing procedure works as follows. Every one of the tens of thousands of grid-cell locations is assigned, in advance, to one of a small number of equally sized groups, using a fixed, repeatable rule based on each cell’s identifying number (so the assignment is not random every time it is run – the same cell always lands in the same group, which makes results reproducible). Then, one group at a time, the procedure sets that group’s cells aside entirely, builds a brand-new copy of the model using *only* the data from the other groups, and then asks that freshly built model to predict outcomes for the group of cells it has never been shown at all. This is repeated once per group, rotating which group is held out each time, so that by the end, every single location in the entire data set has been predicted exactly once by a version of the model that never saw it during training. This procedure is commonly called **cross-validation**, and doing it by holding out entire locations (rather than, say, holding out random individual rows scattered across many locations) is essential here, because holding out scattered rows would let information leak: rows from the very same forest patch, in nearby years, are not independent of each other, so if some rows from a patch were used for training and other rows from that same patch were used for testing, the test would be artificially easy and would overstate how well the model actually generalizes.

A concrete analogy makes the point vivid. Imagine a class studying from a textbook divided into several chapters, and a teacher wants to know whether the students genuinely understand the material or have simply memorized the specific example problems. Handing students the exam questions and the answer key in advance, and then grading them on getting every answer right, would tell the teacher nothing – it only proves memorization, not understanding. A much fairer test is to let each student study all but one of the chapters in depth, and then examine them only on the chapter they have never seen worked examples from. If you rotate which chapter is held back across as many rounds as there are chapters, so that every chapter eventually gets a turn as the “unseen” exam, you get a genuine, well-rounded measure of understanding rather than an

inflated score from an open-book test. This is exactly the logic of held-out-by-location testing: each group of grid cells takes a turn being the “unseen chapter,” and the reported accuracy is always measured on cells the model being scored never got to study.

4.2 The natural noise floor: what seed1 versus seed2 tells us

Recall from Section 2.3 that the original detailed simulator itself is not perfectly repeatable – rerunning the identical historical climate for the identical location, changing only its internal random seed, produces a genuinely different (but equally valid) outcome. This has an important consequence for how we should judge Component S’s accuracy: since even the very “ground truth” data set disagrees with itself between seed1 and seed2, purely because of internal randomness that has nothing to do with climate or location, no predictor that only looks at climate and location (rather than replaying the exact same internal dice rolls) could ever perfectly match either individual run. There is a hard ceiling on how well any such predictor could possibly score, and that ceiling is measurable directly: it is simply how well seed1 predicts seed2 (or vice versa) at the very same locations.

The testing procedure computes this ceiling in two complementary pieces and combines them. First, it checks how internally consistent Component S’s own predictions are with themselves, by splitting the predicted trees at each location into two interleaved halves (essentially, every other predicted tree goes into one half or the other) and checking how closely the typical (middle) value from one half matches the typical value from the other half, across all locations – if the model itself is stable and not just producing noise, its own two halves should agree well with each other. This self-agreement score is then adjusted using a standard statistical correction to account for the fact that using only half the predictions naturally understates how good the full model actually is (this correction is a well-established piece of arithmetic that essentially answers “if I only tested half the data, how much would that alone shrink my apparent accuracy, and can I compensate for that shrinkage” – the same everyday idea as judging a whole cake from tasting only half of it, then mathematically compensating for having tasted less than the whole thing; the reader does not need its details, only that it exists specifically to make the half-versus-half comparison fair). Second, the identical style of comparison is applied to the two independent full simulation runs – seed1 versus seed2 – measuring how reliable a single simulation run is as a stand-in for “the truth,” given that even the truth disagrees with itself between runs.

These two reliability numbers are then combined into a single **ceiling** by multiplying them together and taking the square root of the result. Here is the intuition for why: think of the two reliability numbers as two hazy windowpanes stacked one in front of the other, between you and the true forest – one haze from the emulator not perfectly agreeing with itself, one haze from the original simulator not perfectly agreeing with itself between seed1 and seed2. Looking through both panes together is blurrier than looking through either pane alone, but not simply as blurry as adding the two blurs together. Multiplying the two reliability numbers and then taking the square root is the standard statistical recipe (called a geometric mean) for estimating how clear the combined view is – worse than the clearer pane alone, better than the blurrier pane alone. The result is a single ceiling number: the best possible accuracy Component S could mathematically achieve, given that neither Component S’s own internal consistency nor the original simulator’s own repeatability is perfect. Finally, dividing Component S’s actual, measured accuracy against real held-out data by this ceiling produces what the project calls the **r_center** score. A result at or near 1.0 on this adjusted scale means the model is doing about as well as is mathematically possible, given that nothing in this picture – not the emulator, not the original physical simulation it is copying – is perfectly repeatable in the first place.

4.3 Testing on a climate the model has never seen: the future-climate hold-out

A separate, complementary test asks a different question: has the model learned something that will hold up under a genuinely different climate regime, or has it only learned to reproduce the particular historical climate period it was shown? To check this, the model is trained using only the historical period’s data and then evaluated purely on its ability to predict the future-climate period it has never seen during training (and, as a further check, this is also run in reverse – training on the future-climate period and testing on the historical period). If accuracy holds up reasonably well across this climate switch, it is much stronger evidence that the model has learned a genuine, transferable relationship between environmental conditions and forest demography, rather than having simply memorized surface-level quirks specific to one twenty-year stretch of weather.

5 Understanding the Numbers: What Each Measure Means

Before looking at the actual results, this section explains, one at a time and using everyday analogies, exactly what each accuracy measure used in this report is really asking. None of the definitions below require any equation – each is described purely as a step-by-step recipe.

5.1 R-squared (written R^2)

R-squared answers the question: compared to the laziest possible strategy – always guessing the overall average value, no matter what the actual situation is – how much of the real, location-to-location variation did the model actually manage to explain? The recipe is: take every one of the model’s prediction misses (the difference between what it predicted and what actually happened), square each miss (multiplying it by itself, which makes every miss positive and makes bigger misses count disproportionately more), and add all those squared misses together. Separately, take every real observed value, measure how far it sits from the overall average of all the real values, square each of those distances too, and add them all together (this second total represents how spread-out the real data naturally is, regardless of any model). R-squared is then one minus the ratio of the model’s own total squared misses to that second, baseline total. A value of 1.0 means every single prediction landed exactly on target – the model’s own total is zero. A value of 0.0 means the model’s predictions did no better, overall, than always guessing the plain average. Negative values (though not seen in this report) are possible and would mean the model did *worse* than the laziest average-guessing strategy.

An analogy: imagine a weather forecaster being judged against a rival who never actually looks at the sky and always predicts “today will be exactly the month’s average temperature.” If a real forecaster’s predictions track the true daily ups and downs closely, they will score close to R-squared of 1.0; a forecaster who might as well not have bothered looking at any data will score close to 0.0.

5.2 Root-mean-square error (RMSE)

Root-mean-square error answers a more direct question: on average, how big is a typical miss, expressed in the original, familiar units of the quantity being predicted (for example, in trees per unit area, or in whatever units wood density is measured in)? The recipe: take every prediction miss, square it (so a miss counts as “bad” regardless of whether the model over- or under-guessed, and so that a big miss counts much more heavily than several small ones), average all of those squared misses together into one number, and then take the square root of that average to undo the earlier squaring and return the result to the original, interpretable units.

An analogy: imagine a dart game scored by the squared distance of each throw from the bullseye, rather than the plain distance. A single throw that lands ten inches off the bullseye contributes twenty-five times as much to the score as a throw that lands only two inches off (because ten squared is one hundred, and two squared is only four – one hundred is twenty-five times four). Root-mean-square error is, in effect, the “typical miss size,” but computed in a way that makes a few very bad misses weigh much more heavily than many small, forgivable ones.

5.3 The Kolmogorov-Smirnov comparison (a whole-distribution comparison)

Sometimes we do not want to compare one predicted number against one true number for the same case; instead, we want to compare two entire collections of values against each other – for example, “does the whole spread of predicted wood-density values across many different forest patches look like the whole spread of the real wood-density values across those same patches?” Recall from Section 3.1.3 that a **distribution** simply means an entire collection of values, not one summary number. The Kolmogorov-Smirnov comparison (often abbreviated “K-S”) is a standard way of comparing two whole distributions directly.

The recipe: line up every value in the first collection from smallest to largest, and separately do the same for the second collection. Now walk along the number line, and at every point ask, for each collection separately, “what fraction of that collection’s values have I passed so far?” This traces out a rising staircase-like curve for each collection, starting at zero percent (before the smallest value) and ending at one hundred percent (after the largest value). The Kolmogorov-Smirnov number is simply the single biggest vertical gap between these two rising curves, measured anywhere along the number line. For example: suppose that at a particular wood-density value, forty percent of the predicted values fall below that point but only twenty-five percent of the real values do – that is a fifteen-percentage-point gap at that one spot on the number line. The Kolmogorov-Smirnov number is whichever such gap, anywhere along the line, turns out to be the largest. If the two whole collections of values look essentially alike, their two rising curves will nearly overlap everywhere, and the biggest gap between them will be small, near zero. If the two collections are almost completely separated from each other (imagine one collection’s values are all much smaller than the other’s), the gap will be large, approaching its maximum possible value of one. So a small Kolmogorov-Smirnov number means “these two whole distributions look alike”; a large one means “these two whole distributions look substantially different from each other,” even if no single matched pair of values is being compared directly.

5.4 Wasserstein (“earth-mover”) distance – a concept worth knowing, though not currently computed in this project

This measure is worth explaining because it addresses a real limitation of the Kolmogorov-Smirnov comparison above: K-S only asks *whether* two distributions’ rising curves separate somewhere, but it does not care *how far apart* the values themselves have shifted. The Wasserstein distance, often nicknamed the “earth-mover’s distance,” imagines each distribution as a pile of sand or dirt, with the pile’s shape following how the values are spread out (more sand where values are common, less where they are rare). It then asks: what is the smallest possible total amount of “sand times distance moved” required to physically reshape the first pile into the second one? Two distributions that have shifted only slightly relative to each other would require moving only a little sand a short distance, giving a small Wasserstein number; two distributions that are shifted far apart would require moving the same amount of sand a much longer distance, giving a larger Wasserstein number – *even in a case where the Kolmogorov-Smirnov measure above might treat both situations identically*, because K-S only tracks the biggest gap, not how far the mismatched material has to travel.

This measure is included here purely for conceptual completeness. A note appears in the project’s own code mentioning an intention to compute it without needing extra external software, but on close inspection, no part of the actual codebase currently calculates a Wasserstein number, and none of the result files in this project report one. It should therefore be understood as a documented idea, not a metric that has actually been used to produce any number appearing later in this report.

5.5 Interquartile-range-normalized root-mean-square error (nqrmse)

This measure adapts the root-mean-square error idea (Section 5.2) so that traits measured in very different units and very different natural scales – for instance, wood density versus tree height – can be compared to each other on a level footing.

First, a bit of vocabulary: a **quantile** (also called a **percentile** when expressed on a zero-to-one-hundred scale) is simply the value below which a given fraction of a collection of numbers falls – for example, the twenty-fifth percentile of a collection is the value that is bigger than exactly twenty-five percent of that collection’s values, and smaller than the rest. The **interquartile range** is the gap between the seventy-fifth percentile value and the twenty-fifth percentile value – in other words, the width of the middle half of the data, deliberately ignoring the most extreme, unusual values at either end, which makes it a fairly stable measure of “how spread out is the typical middle of this collection.”

The recipe: pick five landmark points along the distribution – the fifth, twenty-fifth, fiftieth (the exact middle, also called the **median**), seventy-fifth, and ninety-fifth percentiles – for both the real, observed collection of values and the model’s predicted collection of values. Compute a root-mean-square error (as in Section 5.2) using just these five landmark points as the “predictions” and “true values.” Finally, divide that result by the interquartile range of the real observed data, computed as above. The division step is what makes different traits comparable to one another despite having wildly different natural units and typical magnitudes.

An important caveat, stated plainly in the project’s own code comments: this measure can behave oddly for traits with a very “heavy tail” – meaning a trait where most values cluster in a modest range but a small number of individuals have dramatically larger values. Per-tree biomass is an example given directly in the project’s own notes: the middle-half width (the interquartile range) for per-tree biomass is on the order of one to two hundred units, while the gap out at the ninety-fifth-percentile landmark can be roughly twenty times larger still, in the thousands of units. In such a case, a single landmark point far out in the tail can dominate the whole calculation and make the final number look much worse than the model actually performs on the bulk of realistic cases. This means a before-and-after comparison using this measure is only fair if the two things being compared have a genuinely similar overall spread to begin with – otherwise the number can shift for reasons that have nothing to do with the model actually getting better or worse.

5.6 The prediction-spread ratio

The final measure addresses a specific, easy-to-miss failure mode: a model that plays it safe by always guessing close to the overall average, rather than genuinely distinguishing very different locations from one another. First, recall **standard deviation**: this is simply one number that summarizes how scattered a collection of values is around its own average – small if the values all cluster tightly near the average, large if the values are spread widely, some far above and some far below. The prediction-spread ratio is computed by taking the standard deviation of the model’s predictions (looking across many different locations) and dividing it by the standard deviation of the real, true values across those same locations.

An analogy makes the failure mode vivid: imagine a person asked to guess the height of every student in a school, one at a time, who – regardless of whether a given student is actually short or tall – always answers with something very close to the entire school’s average height. Individually, many of their answers might land “close enough” to be scored as reasonably accurate, especially for students near the middle of the height range. But this person has learned nothing about telling individual students apart from each other; their own set of answers barely varies at all, even though the real students vary a great deal. A prediction-spread ratio close to 1.0 means the model’s predictions vary from place to place about as much as reality actually does – it is genuinely distinguishing different locations. A ratio well below 1.0 means the model is flattening real differences toward the middle, quietly hiding a lack of genuine discriminating power behind individually plausible-looking numbers. This is precisely the concern raised later for wood density (Sections 6.2 and 7): if the model’s wood-density predictions vary much less from place to place than real wood density does, it may be quietly playing it safe rather than genuinely telling a dry-savanna tree’s wood apart from a wet-tropics tree’s wood.

6 Results: What the Validation Actually Shows

Before reading the numbers below, it is worth flagging why grid-cell counts differ slightly from table to table in this section – 67,420 (every location on the global grid), 53,699 / 58,496 / 58,588 (the tree-count tables), 52,165 (the per-trait tables). Not every one of the 67,420 global locations has usable living-tree data for every period or every calculation; each table simply reports the exact subset of cells that had enough data for that specific computation, not a sign of data being silently dropped.

The numbers in this section come from the most recent, complete generation of Component S’s training and validation pipeline, referred to internally as generation “t8.” The correction that made the model’s training data include the full set of seven tree species-groups the simulator actually produces, rather than silently missing two of them, was made in an earlier generation, referred to as “t7” (Section 7.1 explains this bug and how it was caught, and defines what “generation t7” refers to). Generation “t8” builds on that same, already-corrected population of trees – the population of trees being counted did not change between t7 and t8 – and refines how two of the environmental input clues (soil moisture and leaf area index, Section 2.5) are computed from the underlying daily physical simulation.

6.1 Tree-count prediction accuracy

Table 1 reports the decision-tree-forest model’s accuracy at predicting how many trees exist, evaluated three ways: on the historical climate period alone, on the future-climate period alone, and pooling both together.

Table 1: Tree-count prediction accuracy (R-squared, unless noted; RMSE in the same units as tree counts).

	Historical	Future climate	Pooled
Rows / grid cells	22,467,348 / 53,699	99,028,310 / 58,496	121,495,658 / 58,588
R-squared, trained-and-tested on same data	0.9827	0.9823	0.9824
R-squared, held-out-by-location	0.9826	0.9823	0.9824
(typical miss size, RMSE)	(0.689)	(0.698)	(0.697)
R-squared, held-out-by-climate-period	0.982	0.9818	—
Per-location-average R-squared	0.9988	0.9989	0.9989

A word on the “rows” figures in the top row of Table 1 first: these are counts of *patch-year* rows – one row per forest patch per year, carrying that patch-year’s total tree count as the target

the model tries to predict – not counts of individual tree records. The underlying per-tree data feeding the rest of this report is considerably larger, as noted in Section 2.6 (roughly 198 million individual tree-year records for the historical period and around 829 million for the future-climate period).

The last row of the table, “Per-location-average R-squared,” measures something subtly different from the rows above it: instead of pooling every patch-year row from every location into one single calculation, it computes a separate R-squared for each grid cell on its own – how well the model tracks that one cell’s year-to-year ups and downs – and then averages those thousands of individual per-cell scores together. It comes out noticeably higher (around 0.999) than the pooled figures above it (around 0.982) because it rewards getting each location’s own year-to-year *pattern* right, rather than requiring every single row, across every location, to land close to its absolute value; a location whose tree count barely changes year to year is easy to track well in this per-location sense, even though its absolute values contribute to the harder pooled comparison above.

Two things stand out from the rest of the table. First, the held-out-by-location accuracy (0.9826 historical, 0.9823 future-climate, 0.9824 pooled) is essentially identical to the accuracy measured on data the model was trained on directly (0.9827, 0.9823, 0.9824) – meaning the model is not simply memorizing the locations it was shown; it generalizes to entirely unseen locations just as well as it fits the ones it studied. Second, the held-out-by-climate-period results (0.982 and 0.9818) are nearly indistinguishable from the held-out-by-location results, which is reassuring evidence that the model has learned a genuine climate-to-demography relationship rather than a historical-period-specific pattern.

One honest gap should be flagged here: unlike the individual traits discussed next, there is currently no equivalent “natural noise-floor ceiling” computed for the tree-count prediction (the seed1-versus-seed2 comparison described in Section 4.2 has not yet been built out for counts). This means that while an R-squared of roughly 0.982 is very high in absolute terms, we cannot yet say precisely how close this is to the best any model could possibly do, the way we can for the individual traits below. This is explicitly listed as still-open work, not a result being hidden.

The numbers in Table 1 are pooled across every grid cell on Earth; Figures 1–5 show the same held-out predictions spread back out across the globe and across individual patches, so a reader can see directly where the model succeeds and where it does not, rather than trusting a single pooled number.

6.2 Trait prediction accuracy – the four live axes

Table 2 reports, for each of the four traits the copula actually assigns during a running simulation, both the raw held-out correlation between predicted and true values (a correlation of 1.0 would mean perfect agreement in how predictions rank relative to each other; 0 would mean no relationship at all) and the noise-floor-adjusted r_center score described in Section 4.2, together with the prediction-spread ratio from Section 5.6. Two of the columns need a short explanation before the table: “Noise floor” is the seed1-versus-seed2 reliability by itself – literally, how well one full simulation run predicts the other, with no involvement from Component S. “Ceiling” folds in one more ingredient on top of that: it also accounts for how self-consistent Component S’s own predictions are with themselves (Section 4.2), which is why Ceiling is always a little higher than Noise floor. “Gap” is simply the plain arithmetic distance still open between what the model achieves today (Raw corr.) and that Ceiling – Ceiling minus Raw correlation.

Reading this table: the r_center column divides the model’s raw accuracy by the ceiling, so a value approaching 1.0 means the model is close to the best achievable performance, while a lower value shows genuine remaining headroom rather than an artifact of the noise floor. By this

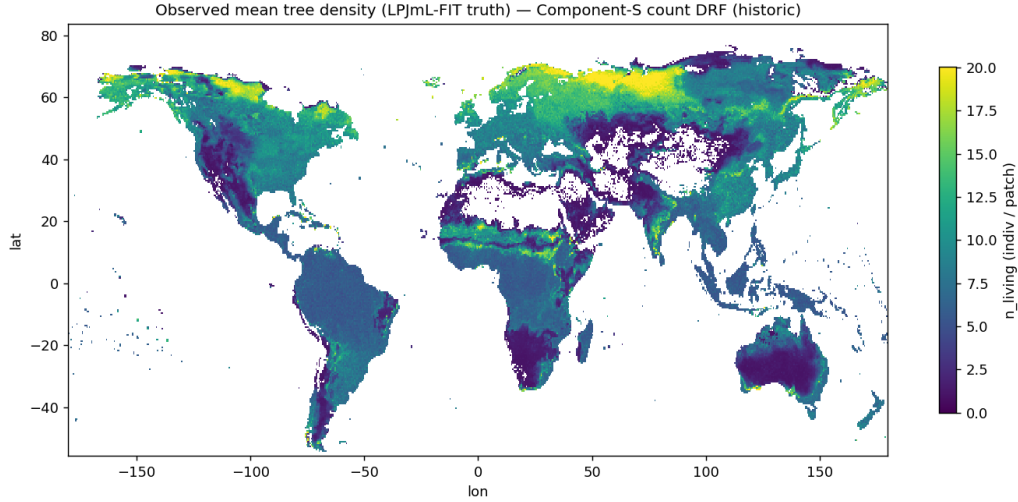


Figure 1: The real, simulated global map this section is trying to reproduce: the average number of living trees per simulated forest plot (“patch”), for every one of the world’s grid cells, as actually produced by the detailed physical simulator over the historical period. Brighter yellow means more trees per plot; dark purple means very few; white areas are locations with no living trees recorded at all (deserts, ice, and similar).

Table 2: Per-trait accuracy on the historical period, 52,165 grid cells (raw correlation, the seed-to-seed noise-floor and ceiling, the resulting gap, the noise-floor-adjusted score, and the prediction-spread ratio).

Trait	Raw corr.	Noise floor	Ceiling	Gap	r_center	Spread ratio
Specific leaf area	0.881	0.973	0.986	+0.104	0.894	0.907
Wood density	0.814	0.937	0.964	+0.150	0.844	0.678
Rooting depth (D95max)	0.791	0.833	0.909	+0.118	0.870	0.714
Drought tolerance (minwscal)	0.945	0.973	0.986	+0.041	0.958	0.970

measure, drought tolerance is the strongest of the four traits (r_center 0.958, meaning it has closed almost all of the achievable gap), specific leaf area is also strong (0.894), and wood density is clearly the weakest of the four (0.844), with the largest raw gap to its ceiling (0.150) and also the lowest prediction-spread ratio of the four (0.678, meaning the model is under-representing how much wood density genuinely varies from place to place, more so than for any other trait). Rooting depth sits in between (0.870), with a similarly modest spread ratio (0.714) – so, of the four traits, two (specific leaf area and drought tolerance) are close to their ceilings today, while the other two (wood density and rooting depth) are the traits with real, quantified room left to improve, and are treated that way in Section 7’s list of open work.

An identically structured measurement, computed slightly differently (per-location held-out correlations across historical, future, and pooled data, plus a Kolmogorov-Smirnov comparison of the whole predicted-versus-true distribution at each location), is shown in Table 3 and tells a consistent story. The last column reports the median (the middle value, once every location’s own Kolmogorov-Smirnov number is lined up in order) per-location Kolmogorov-Smirnov value, pooled across both climate periods.

An earlier generation of this pipeline, referred to as “t7,” produced a similar but distinct set of numbers for the tree-count model (for example, a pooled tree-count R-squared of about 0.9819 rather than the current 0.9824, from before the environmental-clue refinements that define gener-

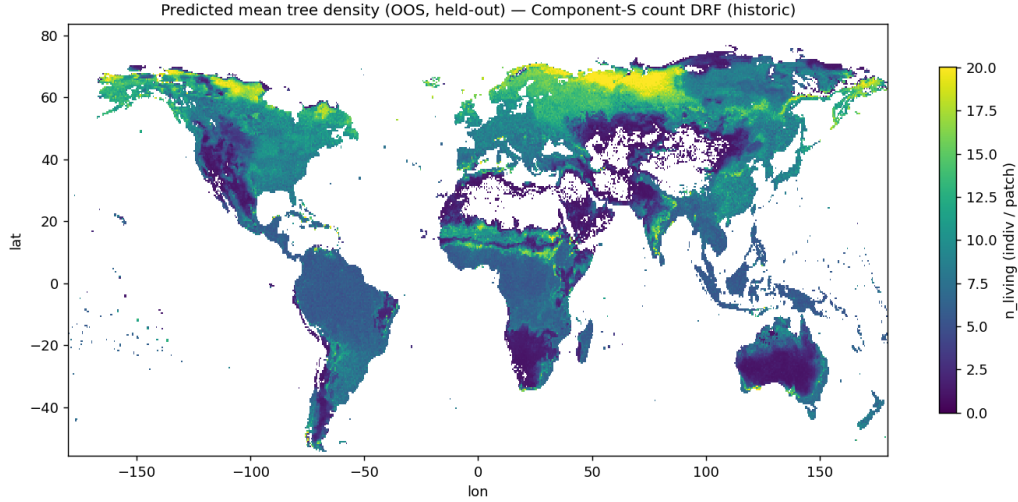


Figure 2: Component S’s own prediction for the exact same quantity, location, and time period as Figure 1 – but every single grid cell shown here was held out of that particular prediction’s training data (Section 4.1), so this is a genuine out-of-sample forecast, not a fit to data the model already knew the answer to. Side by side with Figure 1, the two maps are visually almost indistinguishable, which is the visual counterpart of the 0.9826 held-out-by-location R-squared reported in Table 1.

Table 3: Per-trait held-out correlation across historical, future-climate, and pooled data, and the median per-location Kolmogorov-Smirnov value (pooled).

Trait	Historical corr.	Future-climate corr.	Pooled corr.	Median K-S (pooled)
Specific leaf area	0.880	0.903	0.899	0.173
Wood density	0.812	0.814	0.826	0.129
Rooting depth (D95max)	0.789	0.770	0.776	0.158
Drought tolerance (minwscal)	0.944	0.962	0.967	0.149

ation t8). Those t7 numbers have been superseded by the more complete t8 generation described above and should not be quoted as the project’s current results – they are mentioned here only to show that the project tracks its own progress transparently across generations, rather than reporting only whichever generation looks best.

Figures 6–8 show three complementary views of this same trait-prediction accuracy on 52,516 grid cells (a slightly different cell count from Table 2’s 52,165 because these figures come from a separate, standard validation pass rather than the stricter noise-floor gate; both are legitimate, correctly-labeled measurements on the same underlying model), moving from the whole-world pooled shape of each trait down to individual locations.

6.3 Biomass and tree size – diagnostic-only, and close to the achievable ceiling in this offline check

Recall from Section 3.2.3 that above-ground biomass and tree height are computed through the same copula machinery purely as a diagnostic check, without being fed back into the running simulation. Table 4 reports their accuracy using the same noise-floor-adjusted approach as Table 2. As with every other number in this section, this is an *offline*, table-versus-table comparison: it checks the learned trait-assignment mapping against LPJmL-FIT’s own recorded outcomes for that same year, given that year’s real drivers. It does not, by itself, say anything about how biomass and height would behave if Component S were run recursively – one simulated

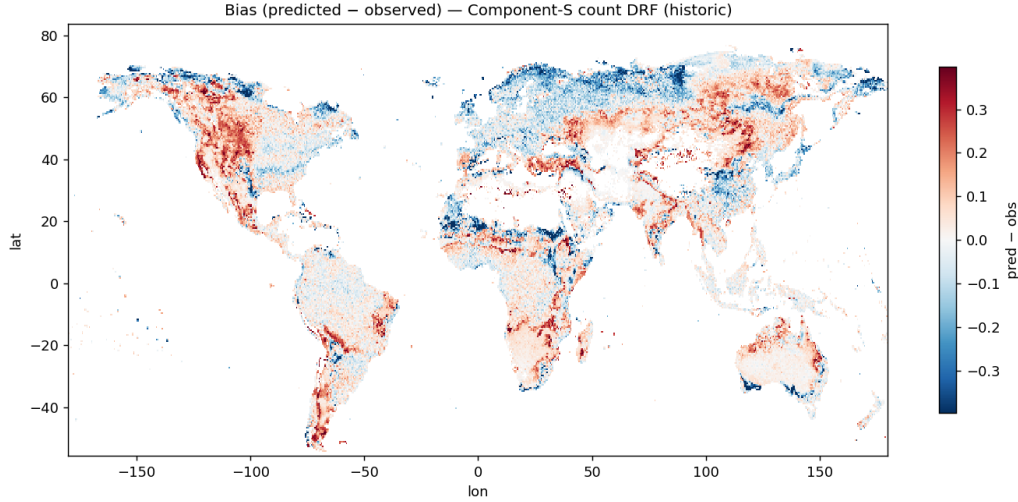


Figure 3: The difference between the two previous maps at every location (predicted minus observed), on a scale centered at zero: white means no detectable error, one color means the model over-predicts tree density there, the other means it under-predicts. Most of the globe is white or very pale, consistent with the high R-squared in Table 1; the clearest exceptions sit right along the boreal tree line, exactly the kind of sharp ecological boundary that is intrinsically hard for any model to place with pixel-perfect precision.

year feeding into the next – inside a live coupled Earth System Model, where small errors could in principle accumulate over time in ways this single-year check cannot detect; that recursive, coupled behavior has not yet been separately demonstrated and remains open work.

Table 4: Diagnostic-only axes: biomass and height, same measures as Table 2.

Axis	Raw corr.	Noise floor	Ceiling	Gap	r_center	Spread ratio
Above-ground biomass	0.864	0.776	0.875	+0.011	0.987	0.822
Height	0.954	0.939	0.967	+0.013	0.986	0.967

Both **r_center** scores here (0.987 and 0.986) sit close to the achievable ceiling in this offline, single-year sense – in plain terms, once you correct for the fact that even two runs of the true physical simulator do not perfectly agree with each other, the model’s biomass and height predictions are about as close to correct as this particular check can measure. As an additional check, a combined “stand biomass” quantity – multiplying the predicted tree count by the predicted biomass per tree – was compared directly against the simulator’s own combined total, achieving a per-location R-squared of 0.931 on the historical period and 0.920 on the future-climate period. These two headline numbers hide a real difference in how consistent they are cell by cell: on the historical period, only about 3.0 percent of cells are more than 10 percent off; on the future-climate period, that rises to about 30.7 percent of cells (traced to a sub-sampling detail in how a small number of very large stands are handled, in a way that affects the future-climate table more than the historical one). The typical (median) cell still agrees closely between predicted and true stand biomass in both periods, which is why this check is judged to pass overall – but the 0.920 future-climate figure should always be read together with that wider cell-to-cell spread, not as a uniformly tight match everywhere. A single combined number spanning both climate periods together was deliberately not computed, because the historical table (about 22 million rows across 20 years) and the future-climate table (about 99 million rows across a longer period) would need to be weighted against each other in a way that has no single obviously correct choice, and combining them without carefully justifying that choice was judged likely to produce

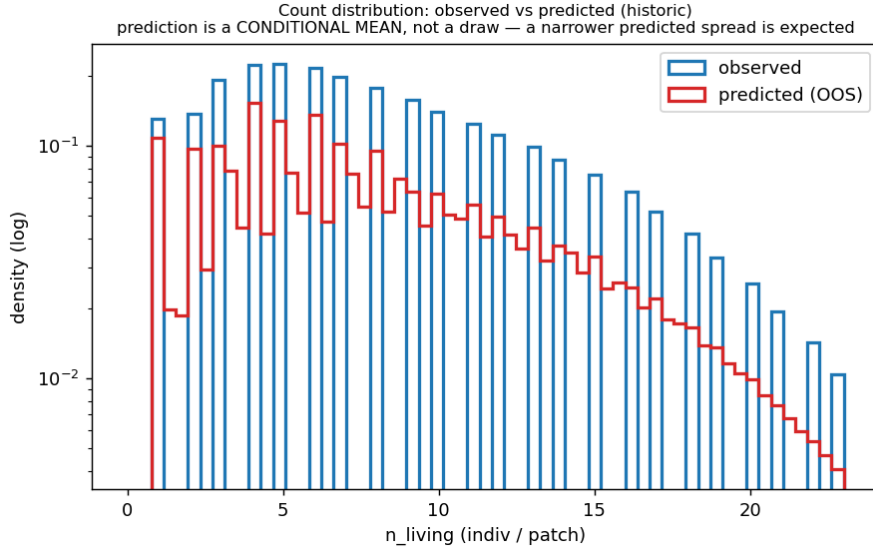


Figure 4: The full distribution (the entire collection of values, not just an average) of tree counts per patch, real versus predicted, pooled across every held-out patch-year in the historical period (the vertical axis is on a logarithmic scale, meaning each gridline up represents a tenfold increase, so both common low counts and rare high counts stay visible on the same plot). The two curves overlap closely across most of the range; the model’s curve runs slightly narrower than reality’s in places, because Component S reports the single most likely outcome for each situation, whereas the real data also includes natural patch-to-patch randomness on top of that average.

a misleading number rather than an informative one – an example of the project declining to report a convenient-looking but invalid statistic (this same discipline appears again in Section 7.2).

6.4 Overall gate result

The project runs a single, formal pass/fail check across all six measured axes (the four live traits plus the two diagnostic axes), using the seed1-versus-seed2 comparison as its basis, across 52,165 grid cells on the historical period. This check is not a repeat of the `r_center` accuracy scores in Table 2 (which correctly remain below 1.0, reflecting real, quantified room to improve) – it is a separate, stricter methodology check confirming that the noise-floor comparison itself is being computed on a fair, apples-to-apples basis (the same population of trees, correctly measured) before any accuracy number is allowed to be trusted at all. The result: every one of the six axes passed this basis check, so the reported pass rate is 6 out of 6, written here as 1.000 – a perfect *pass rate*, not a perfect *accuracy* score. A trait can, and in this case does, score below the theoretical best in Table 2 while still cleanly clearing the bar needed to certify that its accuracy numbers rest on solid ground. This is the project’s most complete, current, formal statement that the accuracy numbers reported above can be trusted – while, as the next section discusses, still flagging specific areas (particularly wood density) as having room to improve rather than being finished.

7 Why We Should Trust These Results

A model’s accuracy numbers are only as trustworthy as the process that produced them. This section describes three separate mistakes the project caught in its own work, presented deliberately as evidence of a rigorous, self-skeptical validation culture – catching and fixing a mistake

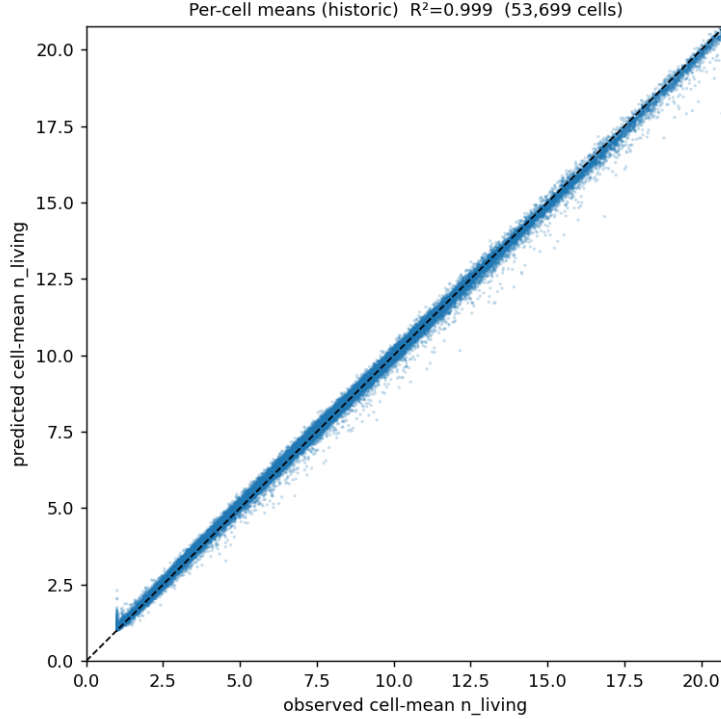


Figure 5: Each of the 53,699 dots is one grid-cell location: its position along the bottom axis is the real, observed average tree count at that location (averaged over all its simulated patches and years); its height is Component S’s held-out predicted average for that same location. The dashed diagonal line marks perfect agreement. Nearly every dot sits almost exactly on that line across the full range from sparse (bottom left) to dense (top right) forest, which is the per-location R-squared of 0.9988 reported in Table 1 made visible.

before it becomes a permanent, uncorrected claim is exactly what a trustworthy scientific process is supposed to look like, not something to be embarrassed about.

7.1 The “which trees count” bug

The detailed simulator distinguishes seven different kinds of tree (various tropical, temperate, and boreal broadleaf and needle-leaf types, plus a boreal larch), in addition to separately modeled grasses and crops. For several months, every table used to train Component S’s production models filtered which trees to include using a list that, by mistake, named only five of these seven tree types, silently excluding the tropical evergreen type and the boreal larch type. This single mistake dropped 32.5 percent of all surviving tree records worldwide from the training data, and made 16.7 percent of all forested grid cells globally – 9,011 out of 54,020, concentrated specifically in the tropical belt and the Siberian larch zone – completely invisible to the emulator during training.

The reason this went unnoticed for so long is itself instructive: the project’s one detailed, hand-checked test location, the German beech forest at Hainich used throughout this report as an example, happens to contain none of the two missing tree types. Every single-location sanity check the project ran during that period passed cleanly, because the bug’s effects were entirely invisible at that one location – a clear illustration of why single-location testing (Section 4.1’s motivation) must eventually be backed up by testing across the full, diverse range of real locations.

The bug was ultimately caught by a deliberate, dedicated check that directly compared the training filter against a full worldwide census of what the simulator had actually produced,

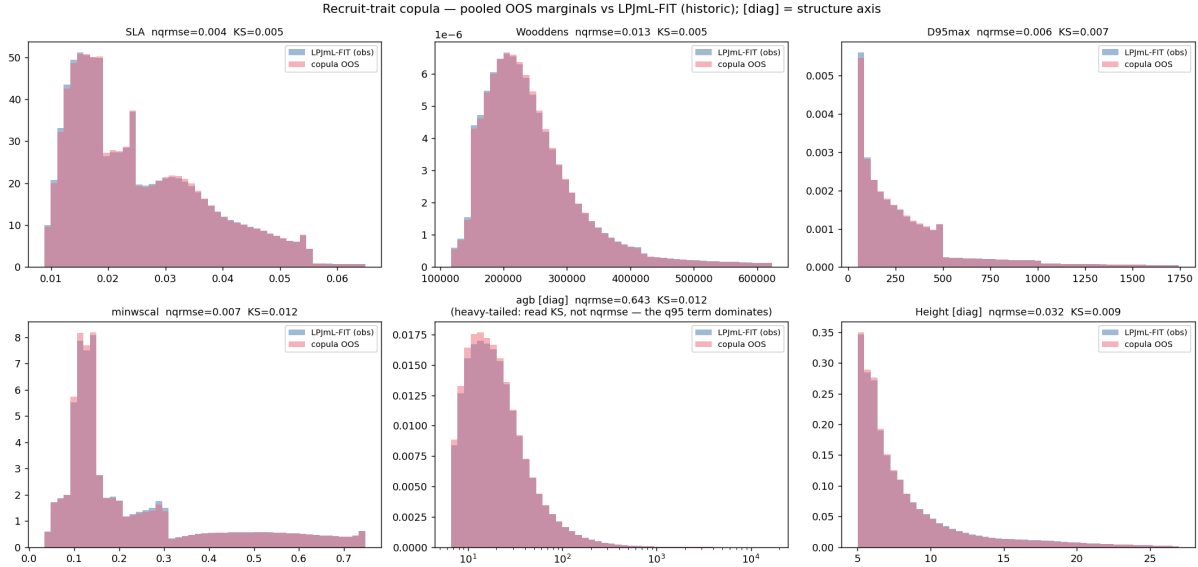


Figure 6: The overall, whole-world distribution of each trait, real versus predicted, pooled across every held-out tree in the historical period. Four panels are the traits actually used by the running model (specific leaf area, wood density [“Wooddens”], rooting depth [“D95max”], drought tolerance [“minwscal”]); the two panels marked “[diag]” are the diagnostic-only biomass and height axes from Section 3.2.3. Every panel reports its interquartile-range-normalized error and its Kolmogorov-Smirnov value; both are small for every trait, and the two curves sit almost on top of each other in all six panels. This is a whole-population comparison, not a location-by-location one – it confirms Component S has learned the right overall shape for every trait globally, but the next two figures are needed to check any single real-world location. The above-ground-biomass panel’s interquartile-range-normalized error (about 0.64) looks alarmingly large purely because that trait is heavy-tailed (Section 5.5’s caveat); its Kolmogorov-Smirnov value (about 0.01) is the trustworthy number for that one panel, and it is small.

and its root cause was traced precisely: the correct, complete list of seven tree types already existed in one file elsewhere in the project, but a second, separately maintained copy of what was supposed to be the identical list – inherited from an older, related project’s configuration and never reconciled – had gone stale. The fix was not merely correcting the wrong list; it was eliminating the duplication itself, so that exactly one authoritative definition now exists, and every other part of the project imports that single definition rather than typing its own copy. This fix is what defines the generation of the pipeline referred to elsewhere in this report as “t7.” The lesson recorded directly in the project’s own decision log is a general one, worth stating plainly: whenever the same fact is maintained in two independent places, silent drift between them is only a matter of time.

7.2 The “basis_ratio” explanation that sounded right but wasn’t

While cross-checking whether the model’s tree-count predictions and its trait/size predictions were mutually consistent with each other, the project computed a particular ratio between the two – specifically, comparing the total number of trees in a patch against the total biomass in that patch divided by the biomass per tree – and, initially, wrote up a plausible-sounding scientific explanation for why that ratio should differ from a value of exactly 1.0: namely, that it revealed a genuine negative relationship between how many trees a patch had and how large those trees tended to be, on average. This explanation was scientifically reasonable on its face, and it made its way into an internal report as a stated conclusion, and from there into a figure caption.

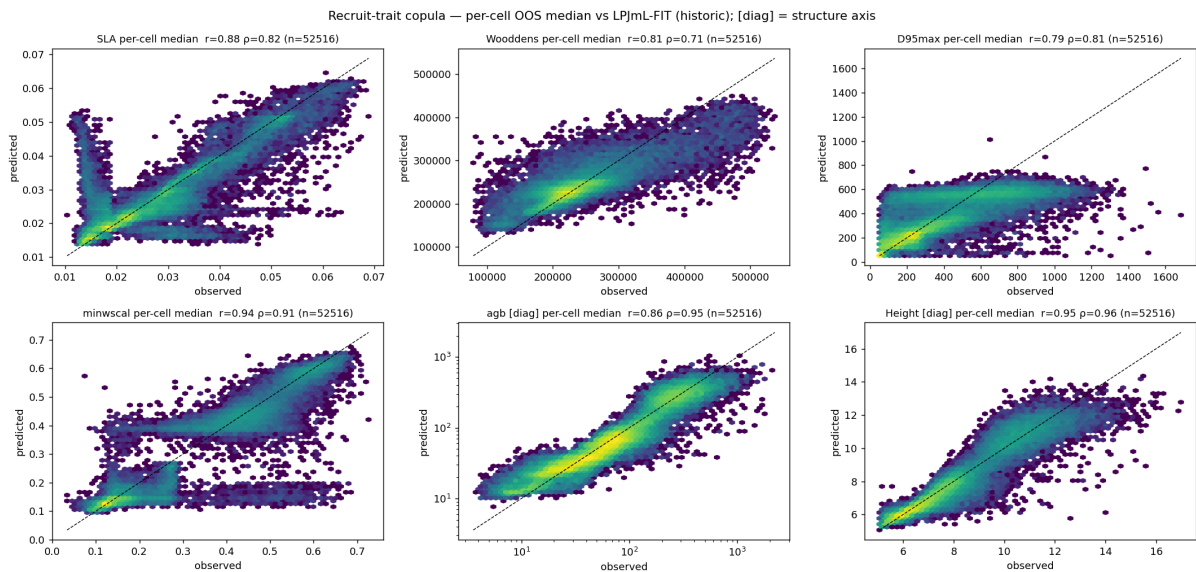


Figure 7: Now zoomed in to individual locations: each dot is one of 52,516 grid cells, comparing the real median value (the middle value once every tree’s trait value at that location is lined up from smallest to largest) of a trait at that location against Component S’s held-out predicted median for that same location, with the dashed line marking perfect agreement. Drought tolerance, biomass and height hug the diagonal tightly across their full range. Wood density (top center) is visibly the noisiest of the four live traits, matching the largest gap and lowest prediction-spread ratio already flagged numerically in Table 2: locations with genuinely very dense wood are, on average, somewhat under-predicted.

On closer mathematical inspection, however, this turned out not to be a real ecological finding at all – it was simple algebra. It is the same situation as computing “total apples” divided by “number of baskets times apples per basket”: by definition those two quantities are always equal, no matter how the apples happen to be distributed among the baskets, so that ratio is guaranteed to equal 1 and tells you nothing whatsoever about how apples are distributed. The tree-count-and-biomass ratio here had exactly the same structure: because the underlying data records one row per individual tree rather than one row per patch, the two competing terms in the ratio cancel out *exactly* whenever the two tables being compared describe the same underlying set of tree records, regardless of any actual ecological relationship. It was, in fact, functioning as a much more mundane (but still useful) check: simply confirming that the two tables being compared genuinely describe the same underlying rows. Four separate reviewers or checks caught this discrepancy – but only after the incorrect explanation had already been written up in an internal report and had reached a figure caption; it was corrected before it was published any further, but not before it had already been written down at least twice. The moral recorded in the project’s own documentation is direct: always work out the mathematics of the specific calculation actually implemented, rather than accepting whichever explanation happens to sound scientifically plausible.

7.3 Rows that silently changed count on their own

A fast big-data processing tool, used in a particular high-speed “streaming” mode (a way of processing a very large table in a continuous flow rather than loading all of it into memory at once) to combine hundreds of millions of rows of simulation output, was found to produce *different* row counts – 99,023,397 rows in one run versus 99,028,310 rows in another run – when run twice on the exact same input data with the exact same instructions, affecting 141 grid cells,

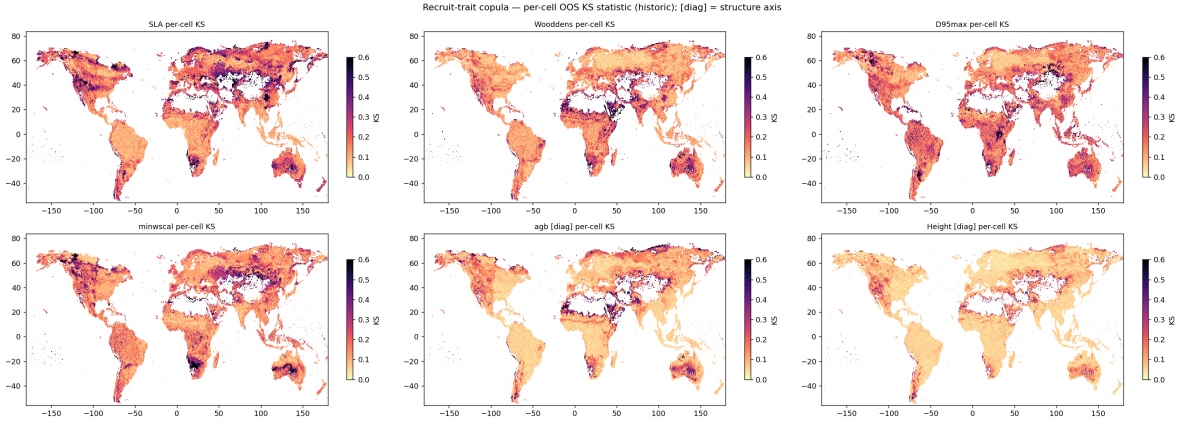


Figure 8: The same per-location comparison, mapped back onto the globe and measured with the Kolmogorov-Smirnov statistic computed separately at every grid cell, showing *where* on Earth each trait’s full local distribution is reproduced well or poorly. Pale colors mean a good local match; dark patches mean a poorer one. Across all six traits, most of the world’s land surface shows a good local match; the harder-to-match patches recur in the same handful of regions across several traits at once, suggesting these are places where Component S’s current set of environmental input clues (Section 2.5) least fully captures the local forest composition, rather than unrelated random noise.

and, more alarmingly, leaving twelve grid cells with outright duplicated location identifiers. This was a genuinely dangerous kind of error, because the standard safety check the project normally relies on – comparing the row count before and after a processing step to catch accidentally dropped data – cannot catch this particular failure mode: duplicated rows can make the before-and-after row counts look perfectly reasonable even while the underlying data has been silently corrupted.

The fix put in place was to stop trusting row counts as the safety check for this kind of large, high-speed processing step, and instead have every such pipeline explicitly verify that each individual location-and-year combination appears in the output *exactly once* – checking the actual identity of the rows, not merely how many of them there are. A separate, more fragile processing step that combined data by matching it against itself was also replaced with a safer alternative technique. This episode, together with the two above, is presented here deliberately: not as a list of embarrassments, but as concrete evidence that the numbers reported in Section 6 have survived a genuinely skeptical, multi-layered checking process, rather than being taken at face value the first time they were computed.

8 Current Limitations and Ongoing Work

Honesty about what is not yet finished is as important as reporting what already works well. As of the results in this report, the following work is explicitly identified as open or incomplete, under an internally tracked next-milestone label referred to as “S2” (short for the second major step in Component S’s ongoing development):

- **Component S currently covers trees only.** Grasses growing in the same patch are handled entirely by the fast physical component F, not by Component S, and giving Component S its own grass-demography prediction is a distinct, not-yet-started piece of future work, not something this report’s results speak to.
- **Every accuracy number in Section 6 is an offline, one-year-at-a-time check, not**

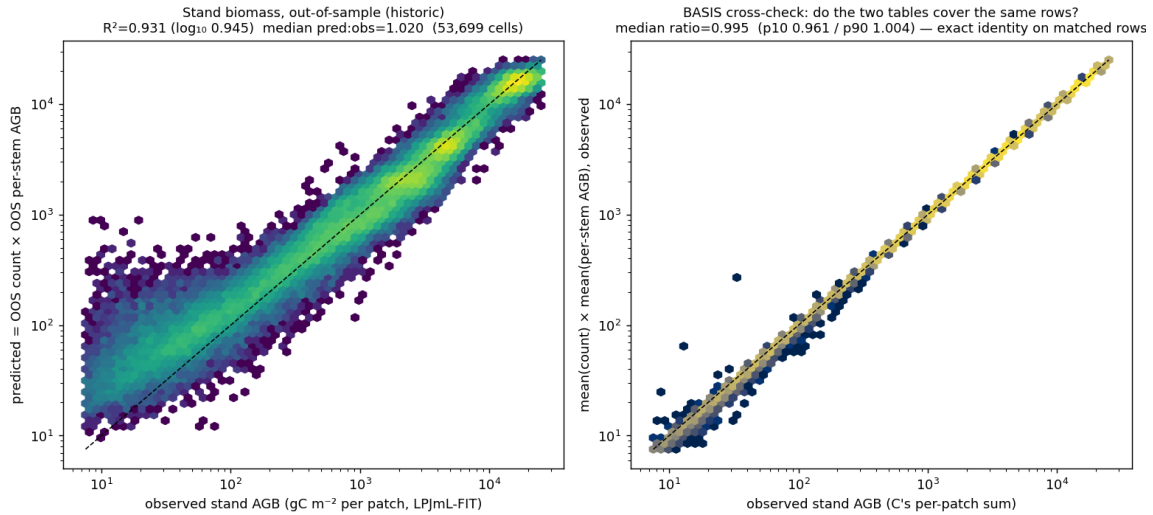


Figure 9: Each dot is one grid cell, comparing the simulator’s own true total stand biomass (bottom axis) against the combined prediction described above – predicted tree count multiplied by predicted biomass per tree (left axis), with the dashed line marking perfect agreement. The historical panel hugs the diagonal tightly, matching the per-location R-squared of 0.931 in the text; the future-climate panel shows more scatter around the line, matching the wider cell-to-cell spread (30.7 percent of cells more than 10 percent off) already flagged in the text above.

a test of recursive, coupled behavior. Component S is designed to be consulted once per year, every year, inside a live running climate simulation, with each year’s output feeding into the next year’s input. The results in this report check how well a single such step matches recorded outcomes when fed the physical simulation’s own real drivers for that year; they do not yet demonstrate how errors might or might not accumulate across many such steps chained together inside an actual coupled run. Building that demonstration is separate, explicitly open work.

- **Wood density remains the weakest of the four live trait predictions.** As shown in Table 2, it has the largest remaining gap to its achievable ceiling (0.150) and the lowest prediction-spread ratio (0.678) of any tracked trait, meaning the model currently under-represents how much wood density genuinely varies from one real-world location to another. The stated near-term target is to close at least half of this remaining gap, and to raise the prediction-spread ratio to at least 0.75. The planned approach is to give the model additional environmental and forest-composition information to work with – this is described as an as-yet-unbuilt, unvalidated next step, not something already completed.
- **Rooting depth is a secondary target**, with a similar though smaller shortfall in its prediction-spread ratio (0.714) that the same forthcoming work is expected to also help address.
- **The two strongest live traits – specific leaf area and drought tolerance – are reported as having comparatively little further room to gain**, so future effort is being deliberately prioritized toward wood density and rooting depth rather than spread evenly across all four traits.
- **A like-for-like natural noise-floor ceiling for the tree-count prediction (Section 6.1) does not yet exist.** While the raw held-out accuracy for counts is very high, the project cannot yet state precisely how close that number is to the best mathematically achievable result, the way it can for the individual traits. Building this comparison is

explicitly listed as still open.

- **A single, combined biomass accuracy number spanning both the historical and future-climate periods together has been deliberately withheld** (Section 6.3), pending a considered choice of how to weight two differently sized tables against each other; reporting a combined number today, without that justified weighting, was judged likely to be misleading rather than informative. The future-climate stand-biomass figure that *is* reported (0.920) also carries a wider cell-to-cell spread than its historical counterpart, as noted in Section 6.3.
- **One of the environmental input clues, the water-stress severity measure** used as one of Component S’s inputs (Section 2.5), is built and maintained by a different part of the wider project team, rather than by the team responsible for Component S itself. It is currently known to disagree with an independent internal check of the same underlying quantity by a substantial, still-unresolved margin – this is a known, quantified physical-fidelity gap in a component Component S depends on but does not control, not a claim that the measure is merely a little noisy.

None of these open items call into question the core, already-validated results in Section 6 within the scope those results actually cover – the tree-count model and two of the four live traits (specific leaf area and drought tolerance) are performing close to their respective achievable ceilings in the offline, one-year-at-a-time checks reported here, and the formal pass/fail basis check across all six measured axes currently passes. The open work is a matter of pushing the two weaker trait axes (wood density and rooting depth) further, extending coverage to grasses, and building out the recursive, coupled-run evidence that complements today’s offline numbers – not a sign that the overall approach is unproven.

9 Conclusion

Component S sets out to answer a focused but genuinely hard question: once a year, given a summary of how a patch of forest fared physically, how many trees of each kind now exist, and what physical traits do the newly established ones carry? Rather than re-running a full, detailed, tree-by-tree physical simulation every single year inside a global climate model – something far too slow to do repeatedly at global scale – Component S learns to imitate that detailed simulation’s own behavior from millions of examples of its output, using two complementary statistical techniques: a large collection (technically called an “ensemble,” meaning simply a group of individual models combined together rather than relied on one at a time) of randomized decision trees for predicting how many trees survive or establish, and a correlated sampling technique (a copula) for deciding what realistic, jointly consistent bundle of traits each new tree receives.

Tested the hard way – on grid-cell locations, and separately on entire climate periods, the model has never been allowed to see during training – Component S currently predicts tree counts and two of its four core live traits (specific leaf area and drought tolerance) at a level close to the best that is mathematically achievable in this offline, one-year-at-a-time sense, given that even two runs of the original physical simulator do not perfectly agree with each other. Wood density and, to a lesser extent, rooting depth remain the clearest areas for improvement, and the project has stated, in concrete and measurable terms, exactly what closing the wood-density gap would look like. Component S today covers trees only, and its numbers describe single-year, offline accuracy rather than proven behavior over many years chained together inside a live coupled simulation – both stated here as open work, not as fine print to be overlooked. Three separate instances of the project catching its own mistakes – an incomplete tree-type filter, a scientifically plausible-sounding but mathematically incorrect explanation, and a subtle data-processing inconsistency

– are presented here not as embarrassments but as the visible evidence of a genuinely rigorous, self-checking process, which is ultimately what should give a reader confidence in the numbers this report presents.

Glossary

Above-ground biomass

The total mass of a tree’s woody and leafy material found above the soil surface – essentially a measure of how much “tree” there is above ground, as opposed to its root system.

Bootstrapping (bootstrap resampling)

A technique for building variety among the trees in a random forest: to build one decision tree, a new training set of the same size as the original is created by drawing examples from the original set at random, allowing the same example to be drawn more than once (“with replacement”), so that by chance different trees end up trained on slightly different mixes of examples.

Ceiling (noise-floor ceiling)

The best possible accuracy any environment-driven predictor could mathematically achieve for a given quantity, computed from how much the two independently re-run copies of the “ground truth” simulation (seed1 and seed2) naturally disagree with each other, combined with how self-consistent the emulator’s own predictions are with themselves. Always a little higher than the Noise floor entry below, because it additionally credits the emulator’s own internal self-consistency.

Copula

A statistical technique for drawing several related quantities together in a way that respects how they tend to rise and fall together in reality (their correlation), while keeping each individual quantity’s own realistic range of values intact; used in this project to assign a jointly consistent set of physical traits to each newly established tree.

Correlation

The tendency of two quantities to move together – rising together (positive correlation) or moving in opposite directions (negative correlation) – without one perfectly determining the other; the everyday example of taller people tending to have larger shoe sizes.

Cross-validation

A testing procedure in which a model is trained repeatedly on different subsets of the data and tested each time on a held-back portion it has never seen, so that the reported accuracy reflects genuine generalization rather than memorization of the training examples.

Decision tree

A simple prediction method that asks a sequence of yes-or-no questions about the available clues (for example, “was drought severity above a certain level?”), routes a case down one branch or another depending on the answer, and, once no more questions remain, predicts the average outcome of whichever training examples ended up following the same sequence of answers.

Distribution

The entire collection of possible or observed values for some quantity, described by its full shape and spread, rather than a single summary number such as an average.

Earth System Model

A large computer simulation of the whole climate system – atmosphere, ocean, ice, and land surface – running together and exchanging information continuously, used to study and project how the climate behaves and changes over time.

Emulator

A fast statistical stand-in model, trained to reproduce the input-output behavior of a much

slower and more detailed physical simulation, so that its behavior can be consulted cheaply without re-running the original slow simulation every time.

Ensemble

A group of individual models combined together and used as one, rather than relying on any single model alone; the random forest used in this project (Section 3.1) is one example of an ensemble.

Gross primary production (GPP)

The total amount of carbon captured by a plant through photosynthesis in a given period, before subtracting the carbon the plant itself burns through respiration.

Interquartile range

The width of the middle half of a collection of values – specifically, the gap between the value at the seventy-fifth percentile and the value at the twenty-fifth percentile – used as a stable measure of a data set’s typical spread that deliberately ignores extreme outlying values.

Kolmogorov-Smirnov statistic (K-S)

A single number, between zero and one, measuring how different two entire distributions look from each other, computed as the largest vertical gap between their two cumulative rising curves; near zero means the two distributions look alike, near one means they are almost completely separated.

Leaf area index

A measure of how much total leaf surface area is layered above a given patch of ground or above an individual tree’s crown – a higher value means denser, more layered foliage.

Leaf economy

A general term for the trade-off among leaf-related traits between building cheap, thin, fast-growing, short-lived leaves versus expensive, thick, tough, long-lived ones; specific leaf area and organ lifespan (both defined in this glossary) are the two leaf-economy traits recorded in this project.

Median

The middle value of a collection of numbers once they are all lined up in order from smallest to largest; also called the fiftieth percentile.

Mortality

The probability, or fraction, of a group of trees that dies within a given year; in this project it is broken down by cause into starvation, old age, drought, and cold-temperature stress.

Net primary production (NPP)

The carbon a plant has left over for growth after subtracting, from its total photosynthetic capture (gross primary production), the carbon it burns through its own respiration.

Noise floor

The natural, irreducible level of disagreement between two equally valid re-runs of the same underlying physical simulation (seed1 versus seed2 in this project), arising purely from the simulation’s own internal randomness rather than from any prediction error; it sets a realistic lower bound on how much disagreement is unavoidable, and therefore an upper bound on achievable accuracy. Distinct from, and always a little lower than, the Ceiling entry above, which additionally folds in the emulator’s own self-consistency.

Organ lifespan trait

An inherited trait describing roughly how long a tree’s leaves and other organs are built to

keep functioning before being shed or replaced; part of the same leaf-economy trade-off as specific leaf area.

Percentile / quantile

The value in a collection below which a given fraction of that collection’s values falls; for example, the value at the twenty-fifth percentile is larger than exactly twenty-five percent of the collection and smaller than the rest.

Photosynthesis

The chemical process by which a plant uses sunlight energy to convert carbon dioxide and water into sugars, releasing oxygen as a byproduct – the fundamental process by which plants capture carbon and build new tissue.

Process-based simulation

A computer model that directly encodes known physical or biological mechanisms (such as photosynthesis or water uptake) to compute its outputs, as opposed to a statistical model that learns patterns purely from data without explicitly encoding the underlying mechanisms.

R-squared (R^2)

A measure, typically between zero and one, of how much of a quantity’s real, location-to-location variation a model’s predictions manage to explain, compared to the simplest possible baseline of always guessing the overall average. It is computed as one minus the ratio of the model’s own total squared prediction misses to that baseline’s total squared misses. A value of one means perfect prediction; a value of zero means the model does no better than always guessing the average.

Random forest

A prediction technique that builds many individual decision trees, each grown from its own randomly resampled subset of the training data and considering only a random subset of the available clues at each branching question, and then averages their individual predictions together to produce a more stable, less overfit overall prediction.

Random seed

A starting number that determines the specific sequence of results a computer’s internal random-number-generating procedure will produce; using the same seed with the same procedure always reproduces the identical sequence, while a different seed produces a different, but equally valid, sequence – analogous to starting a card shuffle from a different initial arrangement.

Respiration

The biological process by which living tissue consumes stored sugars and oxygen to release usable energy, producing carbon dioxide as a byproduct – the metabolic cost a plant pays to stay alive and grow, and the reverse chemical direction of photosynthesis.

Root-mean-square error (RMSE)

A measure of the typical size of a model’s prediction misses, expressed in the original units of the quantity being predicted, computed by squaring every miss, averaging the squared misses, and then taking the square root of that average to return to the original units; squaring means large misses are penalized disproportionately more than small ones.

Rooting depth

How deep into the soil a tree’s root system extends; in this project specifically, the depth within which ninety-five percent of a tree’s roots are found (internally labeled “D95max” when referring to the unconstrained, genetically potential version of this trait), used as an indicator of how much soil-water access a tree can draw on during drought.

Specific leaf area

A leaf-economy trait describing how much leaf surface area a plant builds for a given amount of leaf mass; a high value indicates thin, cheap, fast-growing leaves, while a low value indicates thick, tough, longer-lived leaves.

Spread ratio (prediction-spread ratio)

The ratio of how much a model's predictions vary across different locations to how much the real, true values vary across those same locations; a value near one means the model is genuinely distinguishing different locations from each other, while a value well below one means the model is understating real location-to-location differences by predicting values too close to the overall average.

Standard deviation

A single number summarizing how spread out a collection of values is around its own average – small when values cluster tightly near the average, large when values are widely scattered above and below it.

Statistical emulator

See Emulator.

Transpiration

The process by which a plant draws water up from its roots and releases it as vapor through small pores in its leaves, which is the primary way vegetation moves water from the soil into the atmosphere and is closely tied to how a plant carries out photosynthesis.

Wasserstein distance (“earth-mover’s distance”)

A conceptual measure of how different two distributions are, computed as the minimum total amount of “material moved times distance moved” needed to reshape one distribution's pile of values into the other's shape; unlike the Kolmogorov-Smirnov statistic, it is sensitive to how far apart mismatched values have shifted, not merely whether the distributions separate. This measure is described here for completeness but is not actually computed anywhere in the current project codebase.

Wood density (minwscal is a different trait – see Drought tolerance)

A trait describing how much mass is packed into a given volume of a tree's wood; denser wood is generally associated with greater resistance to drought and structural damage, at the cost of typically slower growth.

Drought tolerance (minwscal)

An inherited trait, internally labeled “minwscal,” describing the minimum water-stress level (Section 2.4) a tree can survive before its growth or survival is compromised; a lower value means the tree can tolerate more severe drought.

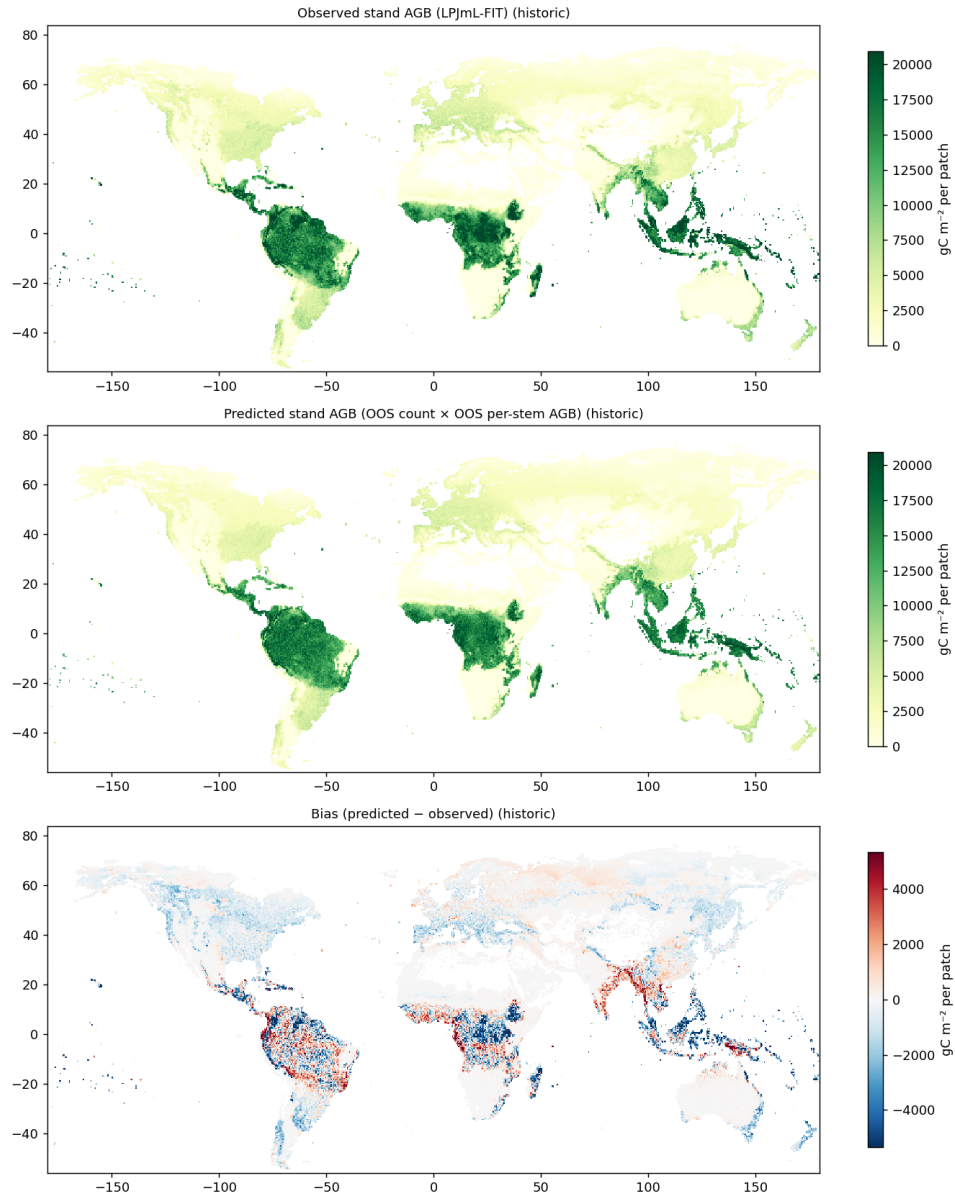


Figure 10: The global map of total stand biomass, real versus predicted, over the historical period – the same kind of side-by-side comparison as Figures 1–3, but for biomass rather than tree count. The overall geographic pattern (dense biomass in the tropics and temperate forests, very little in deserts and tundra) is reproduced closely.